

IOS+ ENGINEERING BODY — DOCUMENT 2 OF 6

Cryptographic Audit & Evidence Fabric Specification

Ed25519 Signing Architecture — JCS Canonicalization — Key Lifecycle —
Triple-Publication Protocol & Evidence Package Format

Document ID:	EB-DOC-002	Classification:	CONFIDENTIAL — Enhanced NDA Required
Version:	1.0	Status:	Authoritative Reference
Date:	May 2026	Series:	IOS+ Engineering Body, Document 2 of 6
Prepared by:	SMEPro Technologies Engineering	Contact:	cmiguez@smepro tech.com
Audience:	Principal Engineers, Security Engineers, Cryptographic Reviewers, NDA-Cleared Auditors		

Table of Contents

Section 0 — Engineering Body Series Context

Section 1 — Cryptographic Design Decisions

1.1 Algorithm Selection: Ed25519

1.2 Canonicalization: JCS per RFC 8785

1.3 Hash Function: SHA-256

Section 2 — Key Lifecycle

2.1 Key Generation

2.2 Key Storage: Software HSM

2.3 Key Rotation

2.4 Emergency Revocation

Section 3 — Triple-Publication Architecture

3.1 Publication Location 1: COS+ Database Key Registry

3.2 Publication Location 2: DNS TXT Records

3.3 Publication Location 3: Deployment Filesystem

3.4 Consistency Verification

Section 4 — Evidence Package Format

4.1 Full Evidence Package Schema

4.2 JCS Canonicalization Procedure

4.3 Verification Procedure

Section 5 — WORM Commitment to COS+

5.1 Commitment Procedure

5.2 Root Hash Publication

Section 6 — Confidence Score Computation

6.1 Composite Score Algorithm

6.2 Confidence Band Mapping

Section 7 — Performance Specifications

Section 8 — Document Lineage

Appendix A — Reference Implementations

Appendix B — RFC References

Appendix C — Threat Model

Appendix D — Glossary

Section 0 — Engineering Body Series Context

The IOS+ Engineering Body (EB) comprises six authoritative internal specification documents covering the complete architecture of the IOS+ system. Each document addresses a distinct subsystem. Together they constitute the canonical reference for principal engineering, security review, and NDA-cleared audit activities.

Doc #	Document Title	Primary Coverage	Status
EB Doc 1	IOS+ System Architecture & Layer Model	Layer stack (L1-L7), middleware-engine, orchestration, service topology	Authoritative
EB Doc 2	Cryptographic Audit & Evidence Fabric Specification <i>(this document)</i>	Ed25519 signing, JCS canonicalization, key lifecycle, triple-publication, Evidence Package format	Authoritative
EB Doc 3	COS+ Database Schema & WORM Enforcement	PostgreSQL schema, append-only triggers, audit_writer role, retention policies	Authoritative
EB Doc 4	RAG Vault & Retrieval Architecture	Vector indexing (HNSW), chunk integrity, semantic search pipeline, retrieval scoring	Authoritative
EB Doc 5	Gate 530 Policy Engine Specification	Dimension definitions, PASS/BLOCK/QUARANTINE logic, tenant regulatory profiles	Authoritative
EB Doc 6	Operational Security & Incident Response	Key rotation ceremony, emergency revocation, consistency check jobs, PagerDuty integration	Authoritative

Scope of This Document

This document — EB Doc 2 — covers the **complete cryptographic architecture of the Evidence Fabric**: the component responsible for producing, signing, committing, and enabling verification of every audit record in the IOS+ system. It is the authoritative reference for the signing key lifecycle, the canonicalization protocol, the Evidence Package binary format, and the triple-publication verification key architecture. All engineering implementations of the Evidence Fabric must conform to this specification. Deviations require a formal engineering change request reviewed by Principal Engineering and Security Engineering.

Section 1 — Cryptographic Design Decisions

1.1 Algorithm Selection: Ed25519

The IOS+ Evidence Fabric uses **Ed25519** (Edwards-curve Digital Signature Algorithm over Curve25519) as its sole signing algorithm. This section documents the design rationale and the alternative algorithms considered and rejected.

Selection Rationale — Ed25519 vs. Alternatives

Property	Ed25519 (Selected)	RSA-PSS- 2048/4096	ECDSA-P256
Signing determinism	Fully deterministic — same output for same message+key	Deterministic (PSS with fixed salt)	Non-deterministic — requires secure random nonce per signature
Signature size	64 bytes	256–512 bytes	64 bytes (variable DER encoding)
Public key size	32 bytes	256–512 bytes	64 bytes (uncompressed)
Weak-key class	None known	Low-entropy key	Small-subgroup

Property	Ed25519 (Selected)	RSA-PSS- 2048/4096	ECDSA-P256
		risk	attacks on some curves
Verification throughput	~70,000/sec per core	~3,000/sec per core	~20,000/sec per core
PRNG dependency in signing	None (deterministic)	Minimal (PSS salt)	Critical — nonce reuse = key compromise
Standard references	RFC 8032, RFC 8037	RFC 8017 (PKCS#1 v2.2)	FIPS 186-4, SEC 1

Deterministic signing is the primary driver of the Ed25519 selection. ECDSA's requirement for a unique, cryptographically secure random nonce per signature operation means that ECDSA signing correctness depends on the quality of the signing system's PRNG at the moment of signing. In a high-throughput audit fabric writing 50,000 records/second, this dependency is an unacceptable systemic risk. Ed25519 eliminates this risk class entirely.

Signature size at scale: At 50,000 evidence records/second sustained write throughput, signature storage cost is a direct function of signature byte length. Ed25519's 64-byte signature represents a 4x–8x improvement over RSA-2048/RSA-4096, directly reducing COS+ storage consumption and index sizes.

Verification speed: The Evidence Fabric's verification endpoint must support batch verification of historical audit records by compliance tools. Ed25519 verification at ~70,000 operations/second per core enables high-throughput batch verification without dedicated verification infrastructure.

Design Principle — Determinism as a Correctness Property

Deterministic signing is not a convenience property — it is a **correctness property for audit infrastructure**. A non-deterministic signature scheme requires the signing system to be trusted to generate good randomness at every signing event. Ed25519 removes that trust requirement from the signing path entirely. Any conforming implementation of Ed25519 with access to the same private key will produce the same signature over the same message. This

property makes Evidence Package authenticity mathematically verifiable, independent of the operational state of the signing system at the time of signature generation.

1.2 Canonicalization: JCS per RFC 8785

To sign a JSON object, the system must produce a **canonical byte sequence** — identical bytes regardless of which implementation, runtime, or serializer produces the JSON. Without canonicalization, two conforming JSON serializers may produce different byte orderings for the same logical object, causing signature verification to fail even for unmodified content.

IOS+ uses **JSON Canonicalization Scheme (JCS)** as specified in RFC 8785. JCS defines the following transformation rules applied to the Evidence Package object:

- **Key ordering:** Object keys sorted lexicographically by Unicode code point. Applied recursively to nested objects.
- **Unicode normalization:** None applied. Raw Unicode code points are preserved as serialized.
- **Number representation:** IEEE 754 double-precision floating-point serialization per the ECMAScript 2019 (ES2019) specification — ensures cross-language consistency for floating-point values such as `confidence_score`.
- **Whitespace:** Compact serialization — no spaces, tabs, or newlines outside string values.
- **String escaping:** Minimal necessary escaping per JSON specification (RFC 8259).

The result is a deterministic, implementation-independent UTF-8 byte sequence that any conforming JCS implementation can reproduce from the same logical object. This is the byte sequence that is signed and verified.

Why Not CBOR or Protobuf Canonicalization

JCS was selected because the Evidence Package is a JSON object throughout its complete lifecycle — created by the evidence-fabric service, stored in COS+ as

JSONB, returned by verification endpoints as JSON, consumed by REST API clients as JSON. Converting to CBOR or protobuf for the signing step would introduce a serialization transformation that is itself a potential source of implementation bugs. JCS canonicalizes the native format with no format conversion required.

Worked Example — JCS Canonicalization

Input object (as received from signing caller, with incidental whitespace and non-lexicographic key ordering):

```
{  "trace_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",  "tenant_id":  
"9b2d3e4f-1a2b-3c4d-5e6f-7a8b9c0d1e2f",  "gate_decision_state": "PASS",  
"confidence_score": 0.91 }
```

JCS canonical form (RFC 8785) — keys sorted lexicographically, compact, no whitespace:

```
{"confidence_score":0.91,"gate_decision_state":"PASS","tenant_id":"9b2d3e4f-  
1a2b-3c4d-5e6f-7a8b9c0d1e2f","trace_id":"f47ac10b-58cc-4372-a567-  
0e02b2c3d479"}
```

Critical

Only the JCS canonical form is signed. The canonical bytes are encoded as UTF-8 and passed directly to the Ed25519 signing operation. The SHA-256 of the canonical bytes is stored in `jcs_canonical_hash` as a convenience field for verification integrity checks. Any verifier must reconstruct the canonical form independently and not rely on any cached or transmitted canonical byte representation.

1.3 Hash Function: SHA-256

SHA-256 is used for **content-addressed references within the Evidence Package**. It is not the signing algorithm — Ed25519 operates on the message directly using an internal hash (SHA-512 of the private key seed in the standard Ed25519 construction). SHA-256 is applied at three points:

Field	Input	Purpose
<code>query_hash</code>	JCS-canonical form of the <code>CanonicalQueryObject</code> (UTF-8 bytes)	Content-addressed identity for the query; enables deduplication

Field	Input	Purpose
		and replay detection
<code>output_hash</code>	Generated output text (UTF-8 encoded, no trailing newline)	Binds the specific output text to the Evidence Package; output modification is detectable
<code>jcs_canonical_hash</code>	JCS canonical bytes of the pre-signing Evidence Package object	Verification convenience — allows verifier to confirm canonical reconstruction before signature check
Chunk integrity (RAG Vault)	Raw chunk bytes at index time	Detects chunk modification between index time and retrieval time

All SHA-256 digests are stored as lowercase hexadecimal strings (64 characters, 256 bits). No truncation is applied.

Section 2 — Key Lifecycle

2.1 Key Generation

Ed25519 key pairs are generated using **libsodium** (`crypto_sign_keypair`). The private key material is a 32-byte seed; the corresponding public key is 32 bytes. Key generation is performed according to a defined key generation ceremony with the following mandatory requirements:

- Key generation is performed on an air-gapped system not connected to any network at the time of generation.
- Private key material is never stored in plaintext at any point. Immediately upon generation, the seed is encrypted under the master key of the software HSM and the plaintext is zeroed from memory.

- Each key generation event is logged with: timestamp (ISO 8601, UTC), generating operator identity (authenticated), public key fingerprint (SHA-256 of public key bytes, hex-encoded), and purpose tag.
- The public key fingerprint is the only artifact retained outside the HSM. Fingerprint mismatch during publication is grounds for immediate key revocation.

Signing Key Record Schema

```
interface SigningKeyRecord {  key_id:                string;  //
  UUID v4 – globally unique key identifier  public_key_bytes:
  string;  // Base64url-encoded 32-byte Ed25519 public key
  public_key_fingerprint:    string;  // SHA-256 hex of public key bytes
  (64 chars)  purpose:        'evidence_signing' |
  'gate_signing';  tenant_id:    string | 'global';  //
  'global' for cross-tenant system key  created_at:    string;
  // ISO 8601 – key generation timestamp  activates_at:
  string;  // ISO 8601 – not used for signing before this time  expires_at:
  string;  // ISO 8601 – not used for new signatures after this time
  revoked_at?:              string;  // ISO 8601 – set only on emergency
  revocation  rotation_predecessor_key_id?: string;  // UUID v4 of previous
  key in rotation chain }
```

Note

The `SigningKeyRecord` contains only the public key bytes. The private key seed never leaves the HSM and is not present in this record. The `key_id` is the linking identifier used in Evidence Packages to select the correct public key for verification.

2.2 Key Storage: Software HSM

The private key seed is stored exclusively in a **software HSM** (HashiCorp Vault with the Transit Secrets Engine). The evidence-fabric service never holds the raw private key in application memory. The signing flow is as follows:

1. The evidence-fabric service computes the JCS canonical bytes of the pre-signing Evidence Package object.
2. The canonical bytes are transmitted to the HSM signing API over a local Unix socket (never over a network interface).

3. The HSM returns the 64-byte Ed25519 signature.
4. The private key seed never leaves the HSM boundary at any point in this flow.

HSM Configuration Parameters

Parameter	Value	Notes
Vault engine	transit	HashiCorp Vault Transit Secrets Engine
Key type	ed25519	Native Ed25519 support in Vault >= 1.6
Key versioning	Enabled	Supports rotation without invalidating historical verifications
Audit logging	All sign operations	Vault audit log captures: caller identity, key version, timestamp, operation type
Access policy	transit/sign permission only	Only the evidence-fabric service account; no other service has sign permission
Export policy	Export disabled	Private key bytes cannot be exported from Vault under any policy

2.3 Key Rotation

Rotation cycle: 90 days, calendar-based (not usage-based). Rotation is initiated by the infrastructure pipeline on a scheduled basis; manual early rotation is permitted for security events.

Rotation Procedure

5. **Step 1:** Generate new Ed25519 key pair in the HSM (new `key_id` assigned, new Vault key version created).
6. **Step 2:** Publish the new public key to all three publication locations — COS+ database, DNS TXT record, and deployment filesystem — using the atomic

update procedure specified in Section 3. All three locations must reflect the new key before activation.

7. **Step 3:** Activate the new key: set `activates_at = NOW() + 1 hour`. The one-hour grace period accounts for clock skew between distributed system components that may query the key registry.
8. **Step 4:** Mark the previous key as expired: set `expires_at = rotation_date`. The previous key is retained in the HSM at its Vault key version — it remains available for **verification of previously-signed records** but issues no new signatures.
9. **Step 5:** Commit a rotation event to the audit log as a signed system event, signed with the **previous key**. The final act of the outgoing key is to sign its own retirement record.
10. **Step 6:** Commit a `key_rotation_record` to COS+ containing both `key_id` values, the cross-signature, and the rotation timestamp.

Verification of Historical Records

Verification of historical Evidence Packages always uses the `signing_key_id` stored in the package to select the correct public key version. There is no requirement to re-sign historical records on rotation. The rotation chain (`rotation_predecessor_key_id`) enables audit reconstruction of the full key lineage for any Evidence Package.

2.4 Emergency Revocation

If a signing key is determined to be compromised, the following emergency revocation procedure is executed immediately:

11. Set `revoked_at` timestamp in the `SigningKeyRecord` in COS+ (propagated to all three publication locations within one TTL cycle).
12. Suspend the evidence-fabric service's HSM signing access for the compromised `key_id` (Vault policy update).

13. All Evidence Packages in COS+ with `signing_key_id` matching the revoked key are flagged: `UPDATE compliance_flags WHERE signing_key_id = <revoked_key_id>`. This is the **only permitted UPDATE operation on audit records**; it is accompanied by a separate revocation audit trail committed with a new signing key.

14. A revocation notice is published to all three key publication locations immediately.

15. New key generation ceremony is initiated per Section 2.1.

The complete emergency revocation incident response procedure, including communication timelines and forensic preservation requirements, is defined in **EB Doc 6**.

Section 3 — Triple-Publication Architecture

Design principle: No single point of failure for key availability. The IOS+ signing public key is published to three independent locations with different trust anchors, access mechanisms, and failure modes. If any one publication location is unavailable, compromised, or attacker-controlled, the remaining two locations provide independent verification capability sufficient to authenticate any Evidence Package.

3.1 Publication Location 1: COS+ Database Key Registry

The primary key registry is the `ios_signing_keys` table in the COS+ database (full schema defined in EB Doc 3, Section 4).

Property	Detail
Access	Any authenticated user with <code>ios_app</code> or <code>audit_reader</code> role
Write mechanism	INSERT new key record only; previous key records are NOT deleted (append-only registry)

Property	Detail
Active key query	<code>SELECT * FROM ios_signing_keys WHERE tenant_id = \$1 AND revoked_at IS NULL AND activates_at <= NOW() ORDER BY activates_at DESC LIMIT 1</code>
Consumers	Compliance review tools, internal audit services, Evidence Fabric verification endpoint, automated consistency check job
Failure mode	COS+ database unavailability — mitigated by DNS TXT and filesystem fallback

3.2 Publication Location 2: DNS TXT Records

The secondary key publication uses DNS TXT records, providing verification capability to any party with DNS query access — including external auditors with no system access to IOS+ or COS+.

Property	Detail
Global key zone	<code>_ios-signing-key.smeprotech.com</code>
Per-tenant key zone	<code>_ios-signing-key-{tenant_id}.smeprotech.com</code>
TXT record format	<code>ios-signing-key-v1 kid={key_id} pk={base64url_public_key} fp={sha256_fingerprint} exp={expires_at_unix}</code>
TTL	300 seconds (5 minutes) — short TTL ensures rapid propagation of rotation events
Update mechanism	DNS API update via infrastructure pipeline, authenticated with TSIG; atomic with HSM rotation step
Security	DNSSEC enabled on zone; TXT record content independently verifiable against database fingerprint
Consumers	External auditors, third-party compliance tools, automated consistency check job

External Auditor Verification Path

DNS publication means **any party on the public internet can retrieve the current IOS+ signing key and verify any Evidence Package**, without requiring access to the IOS+ system, the COS+ database, or any SMEPro-controlled infrastructure. An auditor with a DNS client, a JCS library, and a PyNaCl or libsodium binding has everything required to perform independent cryptographic verification of any Evidence Package. This is a deliberate architectural property — verification independence from SMEPro infrastructure is a compliance assurance guarantee.

3.3 Publication Location 3: Deployment Filesystem

The tertiary key publication uses the deployment host filesystem, providing local verification capability for infrastructure security teams and on-premises auditors.

Property	Detail
Current key path	/etc/ios-plus/signing-keys/ current.pub (symlink)
Versioned key path	/etc/ios-plus/signing-keys/ {key_id}.pub (fixed path per key)
Metadata path	/etc/ios-plus/signing-keys/ {key_id}.meta.json (adjacent JSON metadata per key)
File format	Raw 32-byte Ed25519 public key in binary (.pub); JSON metadata per SigningKeyRecord schema (.meta.json)
Permissions	Readable by any process on the deployment host; writable only by the deployment pipeline service account
Update mechanism	Deployment pipeline writes new key file, then atomically updates symlink: ln -sf /etc/ios-plus/signing-keys/{new_key_ id}.pub /etc/ios-plus/signing-keys/current.p ub (atomic on POSIX filesystems)
Retention	Previous key files retained at their fixed versioned paths for historical

Property	Detail
	verification — symlink update does not remove old files
Consumers	Infrastructure security teams, on-premises auditors with server access, automated integrity monitoring (file hash monitoring)

3.4 Consistency Verification

All three publication locations are verified for consistency by an automated daily job: `key-consistency-check` (full job definition in EB Doc 6). The job performs the following steps:

16. Query COS+ `ios_signing_keys` for active key: retrieve `public_key_bytes` and `public_key_fingerprint`.
17. Query DNS TXT record at `_ios-signing-key.smeprotech.com`: parse `pk` and `fp` fields.
18. Read `/etc/ios-plus/signing-keys/current.pub`: compute SHA-256 fingerprint of raw bytes.
19. Compare all three public key fingerprints. All three must match exactly.
20. On any divergence: emit PagerDuty alert (P1) and commit a signed system event to COS+ flagging the divergence condition.

Security Alert Condition

Divergence between publication locations may indicate: (a) a key rotation that partially succeeded and must be completed or rolled back, or (b) an attempted **key substitution attack** on one of the publication locations. Either condition requires immediate investigation by Security Engineering. The consistency check job is a detective control — it does not prevent divergence but detects it within 24 hours.

Section 4 — Evidence Package Format

4.1 Full Evidence Package Schema

The Evidence Package is the central artifact of the IOS+ audit fabric. It is created at Layer 7 (L7) of the processing pipeline, signed, and committed to COS+ as a WORM record. The complete TypeScript interface definition follows, with field-by-field annotations.

```
interface EvidencePackage {    // — Identity
    trace_id:
string;    // UUID v4 – assigned at L4, globally unique,
// present on all downstream records    evidence_package_id:    string;    //
UUID v4 – assigned at finalization
(L7);    // the WORM record primary key in
COS+    // — Query Provenance (L4)
    query_hash:
string;    // SHA-256 hex of JCS-canonical CanonicalQueryObject;
// enables query deduplication and replay detection    tenant_id:
string;    // UUID v4 – tenant scope of this record    session_id?:
string;    // UUID v4 – present if multi-turn session context
model_fingerprint:    string;    // Model ID + version hash – identifies
inference model    // — Temporal Anchors (immutable at layer of creation)
    ingestion_timestamp:    string;    // ISO 8601 – L1
entry timestamp    l4_timestamp:    string;    // ISO 8601 – Evidence
Package construction start    gate_timestamp:    string;    // ISO 8601 –
Gate 530 decision timestamp    l6_timestamp:    string;    // ISO 8601 –
Context assembly start; null if BLOCK    synthesis_timestamp:    string;    //
ISO 8601 – L7 completion; null if BLOCK    // — Gate 530 (L5)
    gate_decision_id:
string;    // UUID v4 – unique identifier for the gate decision
gate_decision_state:    'PASS' | 'BLOCK' | 'QUARANTINE';
gate_decision_record:    GateDecisionRecord;    // Full GateDecisionRecord
embedded    // (defined in EB
Doc 5)    // — Source Manifest (L6)
    source_manifest:
SourceManifestEntry[] | null;    // null if
gate_decision_state = 'BLOCK';    // array
of retrieved RAG Vault chunks with    //
provenance, chunk_hash, and retrieval rank    // — Output (L7)
    output_hash:
string | null;    // SHA-256 hex of
generated output text    // (UTF-8
encoded, no trailing newline);    // null
if gate_decision_state = 'BLOCK'    confidence_score:    number | null;
// 0.0–1.0 composite score (see Section 6);
// null if gate_decision_state = 'BLOCK'    // — Signing Metadata (L7 –
set last, during FinalizeAndCommit)    signing_key_id:
string;    // UUID v4 – key_id of the SigningKeyRecord used
jcs_canonical_hash:    string;    // SHA-256 hex of the JCS canonical bytes
that    // were signed (verification
convenience field)    ed25519_signature:    string;    // Base64url-encoded
64-byte Ed25519 signature }
```

4.2 JCS Canonicalization Procedure

The following procedure is executed by the `FinalizeAndCommit` function in the evidence-fabric service. Steps are ordered and must not be reordered.

21. **Step 1:** Populate all fields of the `EvidencePackage` object **except** `ed25519_signature` and `jcs_canonical_hash`. These two fields are outputs of the signing operation, not inputs; they must be absent from the object at this step.
22. **Step 2:** Apply RFC 8785 JCS canonicalization to the object: sort all keys lexicographically by Unicode code point (recursively), compact serialization, no whitespace outside string values, IEEE 754 number formatting.
23. **Step 3:** Encode the canonical JSON string as UTF-8 bytes. This byte sequence is the signing input.
24. **Step 4:** Compute SHA-256 of the UTF-8 bytes. Store the hex digest as `jcs_canonical_hash`.
25. **Step 5:** Transmit the UTF-8 bytes to the HSM signing API. Receive the 64-byte Ed25519 signature. Base64url-encode the signature bytes. Store as `ed25519_signature`.
26. **Step 6:** Set `jcs_canonical_hash` and `ed25519_signature` in the package object. The package is now complete and signed.
27. **Step 7:** Proceed to WORM commitment (Section 5). The signed package must not be modified after Step 6.

4.3 Verification Procedure

The following procedure may be executed by any party — internal or external — with access to an Evidence Package and any one of the three publication locations. No SMEPro system access is required beyond retrieving the package itself.

28. **Step 1:** Retrieve the Evidence Package (from the IOS+ verification endpoint, or directly from COS+ with `audit_reader` role). Confirm the object is valid JSON.
29. **Step 2:** Extract the value of `signing_key_id` from the package.

30. **Step 3:** Retrieve the public key for the extracted `signing_key_id` from any of the three publication locations: COS+ database, DNS TXT record, or deployment filesystem. Confirm the `key_id` matches and the key is not revoked (`revoked_at` is null).
31. **Step 4:** Reconstruct the pre-signing form of the package: **remove** the `ed25519_signature` and `jcs_canonical_hash` fields from the object. Retain all other fields unchanged.
32. **Step 5:** Apply JCS canonicalization (RFC 8785) to the reconstructed object to produce canonical UTF-8 bytes.
33. **Step 6:** Compute SHA-256 of the canonical bytes. Compare the hex digest to the stored `jcs_canonical_hash` value. If they do not match, the canonical reconstruction has failed — abort verification and report an implementation error.
34. **Step 7:** Verify the Ed25519 signature:
`crypto_sign_verify_detached(signature_bytes, canonical_bytes, public_key_bytes)`. The signature bytes are obtained by Base64url-decoding the `ed25519_signature` field.
35. **Step 8:** Interpret the result: if verification returns `true`, the package is **authentic and unmodified** since signing. If verification returns `false`, the package has been tampered with or the wrong key was used.

Reference Implementation — Python (PyNaCl + python-jcs)

```
import base64 import hashlib import json import jcs from nacl.signing import
VerifyKey from nacl.encoding import RawEncoder def
verify_evidence_package(package: dict, public_key_b64url: str) -> bool:
"""    Verify an IOS+ Evidence Package signature.    Args:
package:    The Evidence Package dict (as parsed from JSON).
public_key_b64url:    Base64url-encoded 32-byte Ed25519 public key
(retrieved from COS+ registry, DNS TXT, or filesystem).    Returns:
True if the signature is valid and the package is unmodified.    Raises:
nacl.exceptions.BadSignatureError if the signature is invalid.
AssertionError if the canonical hash does not match (implementation error).
"""    # Step 1: Extract and remove signing fields    sig_b64url =
package.pop('ed25519_signature')    stored_hash =
package.pop('jcs_canonical_hash')    # Step 2: JCS canonicalization (RFC
8785)    canonical_bytes = jcs.dumps(package)    # returns bytes (UTF-8)
# Step 3: Verify canonical hash (integrity check before sig verification)
computed_hash = hashlib.sha256(canonical_bytes).hexdigest()    assert
```

```
computed_hash == stored_hash, (          f"Canonical hash mismatch: expected
{stored_hash}, got {computed_hash}"      )      # Step 4: Decode public key
and signature      pub_key_bytes = base64.urlsafe_b64decode(public_key_b64url
+ '==')      sig_bytes      = base64.urlsafe_b64decode(sig_b64url + '==')
# Step 5: Verify Ed25519 signature (raises nacl.exceptions.BadSignatureError
if invalid)      vk = VerifyKey(pub_key_bytes, encoder=RawEncoder)
vk.verify(canonical_bytes, sig_bytes)      return True
```

Note — Library Compatibility

The reference implementation uses `python-jcs` (RFC 8785 conformant) and `PyNaCl` (libsodium bindings). Equivalent implementations are available in Node.js (`canonicalize + tweetnacl`), Go (`go-jose/jcs + crypto/ed25519`), and Java (`nimbus-jose-jwt`). All must produce identical canonical bytes for the same input object. See Appendix A for verified library references.

Section 5 — WORM Commitment to COS+

5.1 Commitment Procedure

After signing, the `FinalizeAndCommit` function performs the WORM commitment to COS+ using the `audit_writer` role. The `audit_writer` role is granted INSERT-only permissions on audit tables; UPDATE and DELETE are not granted and are additionally blocked by triggers defined in EB Doc 3.

36. **Step 1:** Establish connection to COS+ as the `audit_writer` role.
37. **Step 2:** INSERT the complete signed Evidence Package into the `evidence_packages` table. Full schema is defined in EB Doc 3, Section 2.
38. **Step 3:** INSERT the `gate_decision_record` into the `gate_decisions` table, linked by `evidence_package_id`.
39. **Step 4:** INSERT each entry of `source_manifest` into the `evidence_source_manifest` table (one row per RAG Vault chunk). No-op if `source_manifest` is null (BLOCK decision).

40. **Step 5:** All three INSERT operations are executed within a **single database transaction**. Either all succeed or none are committed — partial commitment is not possible.
41. **Step 6:** On transaction commit: return `evidence_package_id` to the middleware-engine. The Evidence Package is now a permanent WORM record.

WORM Enforcement

The `audit_writer` role cannot UPDATE or DELETE any row in the audit tables under any circumstance. The trigger defined in EB Doc 3, Section 3.2 raises a PostgreSQL exception on any UPDATE or DELETE attempt, regardless of the calling role. The sole exception is the emergency revocation flag UPDATE described in Section 2.4, which uses a dedicated `revocation_writer` role with a narrowly scoped UPDATE permission limited to the `compliance_flags` column only.

5.2 Root Hash Publication

Every 60 seconds, the `evidence-root-publisher` background service computes a **Merkle root** of all Evidence Packages committed in the preceding 60-second window. This provides a verifiable checkpoint that enables detection of record insertion or deletion even without examining individual records.

The Merkle tree is constructed as follows:

- Leaf nodes: `SHA-256(evidence_package_id || ed25519_signature)` for each package in the window, ordered by `synthesis_timestamp` ascending.
- Internal nodes: standard binary Merkle tree construction (SHA-256 of concatenated child hashes).
- The root hash is signed with Ed25519 using the active evidence signing key.

The signed root hash is published to:

42. COS+ `merkle_roots` table (INSERT via `audit_writer` — WORM record)

43. DNS TXT record: `_ios-merkle.smeprotech.com` (updated via DNS API; TTL 60 seconds)

An auditor may verify the completeness of any 60-second window by reconstructing the Merkle tree from the `evidence_packages` table and comparing the computed root hash against the published signed root. A mismatch indicates record insertion, deletion, or modification within that window.

Section 6 — Confidence Score Computation

6.1 Composite Score Algorithm

The `confidence_score` is a floating-point composite in the range [0.0, 1.0] computed at L7 immediately prior to signing. It is signed into the Evidence Package and cannot be recomputed or modified after commitment.

Composite Formula

```
confidence_score = (w1 × source_freshness_score) + (w2 ×  
semantic_distance_score) + (w3 × gate_passage_score)  
+ (w4 × retrieval_rank_score)
```

Default Weights

Weight	Component	Default Value	Notes
w1	source_freshness_score	0.30	Configurable per tenant regulatory profile
w2	semantic_distance_score	0.25	Configurable per tenant regulatory profile
w3	gate_passage_score	0.25	Configurable per tenant regulatory profile
w4	retrieval_rank_score	0.20	Configurable per tenant regulatory profile

Component Definitions

Component	Formula	Description
source_freshness_score	$\max(0.0, 1.0 - (\text{days_since_indexed} / \text{max_acceptable_age_days}))$	Measures recency of indexed source material. Floored at 0.0; never negative. <code>max_acceptable_age_days</code> is tenant-configured.
semantic_distance_score	$1.0 - \text{mean}(\text{cosine_distance for all chunks in source_manifest})$	Measures semantic relevance. Cosine distances are computed by the RAG Vault HNSW search at retrieval time and stored in the source manifest entries.
gate_passage_score	1.0 (clean PASS) / 0.85 (any WARN-band dimension) / 0.0 (BLOCK — not used)	Encodes the Gate 530 decision quality. BLOCK packages do not carry a confidence score (null).
retrieval_rank_score	$\text{mean}(1 / \text{rank for each chunk in source_manifest})$	Mean Reciprocal Rank (MRR) across all retrieved chunks. Higher-ranked chunks contribute more to this component.

6.2 Confidence Band Mapping

The floating-point `confidence_score` is mapped to a categorical `confidence_band` stored as a separate column in COS+ for query efficiency.

Score Range	confidence_band	Compliance Disposition
0.85 – 1.00	HIGH	No automatic compliance flag; standard audit trail applies
0.65 – 0.84	MEDIUM	No automatic flag; may trigger enhanced review in high-sensitivity tenant profiles
0.00 – 0.64	LOW	Flagged in COS+ <code>compliance_flags</code> for

Score Range	confidence_band	Compliance Disposition
		compliance review; not automatically blocked unless tenant regulatory profile specifies a minimum confidence threshold in Gate 530 dimension 70

Section 7 — Performance Specifications

The following performance specifications are measured on commodity cloud-hosted hardware (4 vCPU, 8 GB RAM) under sustained load equivalent to 50,000 evidence records/second peak write throughput. All latency measurements are for the evidence-fabric service in isolation; end-to-end L1-L7 pipeline latency is covered in EB Doc 1.

Operation	Latency P50	Latency P99	Throughput	Notes
JCS canonicalization (RFC 8785)	0.4 ms	1.2 ms	—	Measured on median Evidence Package size (2.1 KB). CPU-bound; scales linearly with package size.
Ed25519 signing (HSM round-trip)	0.8 ms	2.1 ms	—	Includes HSM API call over local Unix socket. Dominant latency is socket I/O, not cryptographic computation.
SHA-256 (query_hash +	0.1 ms	0.3 ms	—	CPU-bound; negligible contribution

Operation	Latency P50	Latency P99	Throughput	Notes
output_hash combined)				to total latency.
WORM commit to COS+ (3-table transaction)	4 ms	11 ms	50,000 records/sec peak	PostgreSQL INSERT over local network. Bottleneck is fsync on WAL write; connection pooling (PgBouncer) required at peak throughput.
Total FinalizeAnd Commit	6 ms	15 ms	—	Sum of above operations. Within the L7 25 ms latency budget defined in EB Doc 1.
Ed25519 batch verification	—	—	~70,000 verifications/sec per core	Compliance tool verification path; does not affect inference pipeline latency.
Root hash publication (Merkle)	—	—	Every 60 seconds	Background service; no impact on inference path. Merkle tree construction at 50K records/window is $O(n \log n)$ in memory.

Section 8 — Document Lineage

EB Doc 2 exists within the following dependency and reference graph within the Engineering Body series:

Relationship	Direction	Detail
EB Doc 2 → EB Doc 1	Implements	The evidence-fabric package (defined in EB Doc 1, Section 4) implements the cryptographic specification in EB Doc 2. EB Doc 1 is the authoritative source for layer orchestration; EB Doc 2 is authoritative for signing behavior within that architecture.
EB Doc 3 → EB Doc 2	Receives	The COS+ schema (EB Doc 3) defines the tables that receive the WORM commitments described in Section 5 of this document. The <code>evidence_packages</code> table schema in EB Doc 3 must be consistent with the <code>EvidencePackage</code> interface defined in Section 4.1.
EB Doc 5 → EB Doc 2	Provides	The Gate 530 Policy Engine (EB Doc 5) produces the <code>GateDecisionRecord</code> embedded in the Evidence Package. The <code>gate_decision_state</code> values (PASS, BLOCK, QUARANTINE) are defined in EB Doc 5.
EB Doc 6 → EB Doc 2	Operationalizes	EB Doc 6 defines the operational procedures for: key rotation ceremony, emergency revocation incident

Relationship	Direction	Detail
		response, and the key-consistency-check job described in Section 3.4 of this document.

Appendix A — Reference Implementations

The following libraries have been verified as RFC 8785 (JCS) conformant against the official test vectors. All implementations below may be used to construct a verification toolchain without SMEPro system access.

Language	JCS Library	Ed25519 Library	RFC 8785 Verified
Python	<code>python-jcs</code>	PyNaCl (libsodium bindings)	Yes — against RFC 8785 Appendix B test vectors
Node.js	<code>canonicalize</code>	<code>tweetnacl</code> or <code>@noble/ed25519</code>	Yes — against RFC 8785 Appendix B test vectors
Go	<code>go-jose/jcs</code>	<code>crypto/ed25519</code> (stdlib)	Yes — against RFC 8785 Appendix B test vectors
Java	<code>nimbus-jose-jwt</code> (JCS support)	Bouncy Castle Ed25519 implementation	Yes — against RFC 8785 Appendix B test vectors
Rust	<code>serde_json</code> with custom JCS comparator	<code>ed25519-dalek</code>	Partial — community-verified; not officially tested by SMEPro

Note

When using any JCS library, verify against the RFC 8785 Appendix B test vectors before using in a production verification toolchain. In particular, floating-point number serialization (IEEE 754 / ES2019 rules) is the most common source of non-conformance in partial implementations.

Appendix B — RFC and Standards References

Reference	Title	Relevance to EB Doc 2
RFC 8032	Edwards-Curve Digital Signature Algorithm (EdDSA)	Normative algorithm specification for Ed25519 signing and verification
RFC 8785	JSON Canonicalization Scheme (JCS)	Normative specification for Evidence Package canonicalization (Section 1.2, Section 4.2)
RFC 8037	CFRG Elliptic Curves for JOSE	Specifies Ed25519 key representation in JOSE/JWK contexts; governs <code>public_key_bytes</code> encoding
RFC 6962	Certificate Transparency	Reference architecture for Merkle tree construction used in Section 5.2 root hash publication
RFC 8259	The JavaScript Object Notation (JSON) Data Interchange Format	Baseline JSON specification; governs string escaping rules in JCS
NIST SP 800-57 Part 1 Rev 5	Recommendation for Key Management (General)	Key lifecycle guidance governing rotation cycle (90 days), key purpose tags, and revocation procedures (Section 2)

Appendix C — Threat Model

The following threat model covers the primary adversarial scenarios considered in the design of the Evidence Fabric cryptographic architecture. For each threat, the mitigation is specified with reference to the controlling section of this document.

Threat	Attack Vector	Mitigation in EB Doc 2 Architecture	Residual Risk
Tampered Evidence Package	Attacker modifies fields in a stored Evidence Package after signing	Ed25519 signature verification fails deterministically for any modified package. Detectable by any verifier with the public key. (Section 4.3)	Low — cryptographic guarantee; no practical attack against Ed25519 with correct public key
Signing key compromise	Private key seed exfiltrated from HSM	HSM export disabled; key never in application memory. Triple-published keys enable immediate revocation. Rotation chain limits scope of compromise to 90-day window. (Sections 2.2, 2.4, 3)	Medium — HSM compromise is a catastrophic but detectable event; mitigated by revocation procedure in EB Doc 6
DNS TXT spoofing / substitution	Attacker substitutes a different public key in the DNS TXT record	DNSSEC enabled on the DNS zone; cross-check against database and filesystem by daily consistency job. DNS-only verifiers should cross-check fingerprints	Low with DNSSEC; medium without — verifiers should cross-check

Threat	Attack Vector	Mitigation in EB Doc 2 Architecture	Residual Risk
		against a second source. (Sections 3.2, 3.4)	
Key publication divergence	Partial rotation failure or targeted substitution on one publication location	Daily consistency check job compares all three publication locations and alerts on any fingerprint divergence. (Section 3.4)	Low — detected within 24 hours maximum; rotation procedures include atomic publication steps
Replay of old package as new	Attacker presents an authentic historical package as a current response	Every package contains <code>trace_id</code> , <code>tenant_id</code> , <code>l4_timestamp</code> , and <code>synthesis_timestamp</code> — all signed into the package. Replay is detectable by timestamp and <code>trace_id</code> verification. (Section 4.1)	Low — replay is detectable; does not constitute signature forgery
Insider deletion of records	Privileged database user deletes or modifies Evidence Package rows	WORM enforcement at database layer: <code>audit_writer</code> role is INSERT-only; append-only trigger raises exception on UPDATE/DELETE for any role. Merkle root publication enables detection of deletion between checkpoint windows. (Section 5, EB Doc 3)	Low — multi-layer enforcement; Merkle root provides cryptographic deletion detection
Signing of malformed	Bug in evidence-fabric produces	Schema validation is enforced before	Low — validation gate before

Threat	Attack Vector	Mitigation in EB Doc 2 Architecture	Residual Risk
package	and signs an incomplete Evidence Package	canonicalization step. Missing required fields cause signing to abort and raise an error event (not a signed package). (Section 4.2, Step 1)	signing; malformed packages are not committed

Appendix D — Glossary

Term	Definition
Ed25519	An Edwards-curve Digital Signature Algorithm (EdDSA) instantiated over Curve25519, producing 64-byte deterministic signatures. The signing algorithm used throughout the IOS+ Evidence Fabric.
EdDSA	Edwards-curve Digital Signature Algorithm. A family of signature algorithms based on twisted Edwards curves. Ed25519 is the Curve25519 instantiation of EdDSA.
Curve25519	An elliptic curve designed by Daniel J. Bernstein, optimized for use in Diffie-Hellman key exchange (X25519) and digital signatures (Ed25519). Offers strong security properties and no known weak-key classes.
JCS	JSON Canonicalization Scheme. A deterministic JSON serialization algorithm defined in RFC 8785 that produces the same byte sequence for any logically equivalent JSON object, regardless of serializer implementation.
RFC 8785	The IETF RFC specifying the JSON

Term	Definition
	Canonicalization Scheme (JCS). The normative reference for canonicalization in EB Doc 2.
SHA-256	Secure Hash Algorithm 256-bit, producing a 32-byte (256-bit) digest. Used for content-addressed references within the Evidence Package (query_hash, output_hash, jcs_canonical_hash, chunk integrity hashes).
WORM	Write Once Read Many. A data storage paradigm in which records, once written, cannot be modified or deleted. Enforced in COS+ through role-based permissions and database triggers (EB Doc 3).
HSM	Hardware Security Module (or, in IOS+, Software HSM — HashiCorp Vault Transit Engine). A dedicated cryptographic boundary that stores private keys and performs signing operations without exposing key material to application processes.
Merkle root	The root hash of a Merkle tree — a binary tree of cryptographic hashes where each leaf is the hash of a data record and each internal node is the hash of its children. A Merkle root commits to the complete set of records in the tree; any insertion, deletion, or modification changes the root.
canonical serialization	A serialization that produces a unique, deterministic byte sequence for any given logical object, independent of the serializer implementation. Required for signature schemes applied to structured data formats.
signature verification	The cryptographic operation that, given a message, a signature, and a public key, returns true if and only if the signature was produced over that message using the corresponding private key.
key rotation	The periodic replacement of a signing key pair with a new pair, while

Term	Definition
	retaining the old public key for verification of previously-signed records. Defined in Section 2.3.
key fingerprint	SHA-256 of the raw public key bytes, encoded as a 64-character lowercase hex string. Used to identify and compare public keys across publication locations without transmitting the full key bytes.
triple-publication	The IOS+ architecture of publishing the current signing public key to three independent locations (COS+ database, DNS TXT record, deployment filesystem) to eliminate single points of failure for key availability. Defined in Section 3.
signing_key_id	A UUID v4 that identifies the specific SigningKeyRecord used to sign an Evidence Package. Stored in the package and used by verifiers to select the correct public key from any publication location.
evidence_package_id	A UUID v4 assigned at L7 finalization. The primary key of the WORM Evidence Package record in COS+. Globally unique across all tenants.
trace_id	A UUID v4 assigned at L4 entry to an inference pipeline execution. Propagated to all downstream records (gate decision, source manifest, Evidence Package) as the correlation key for the complete audit trail of a single inference request.
confidence_score	A floating-point composite in [0.0, 1.0] representing the computed quality of the IOS+ response, considering source freshness, semantic relevance, gate passage quality, and retrieval rank. Defined in Section 6. Signed into the Evidence Package and immutable after commitment.
confidence_band	A categorical label (HIGH / MEDIUM / LOW) derived from the <code>confidence_score</code> float, stored in COS+ as a separate indexed column for query

Term	Definition
	efficiency. Mapping defined in Section 6.2.